StaffSync – Modern Human Resource Management Solution

Full Stack Development with MERN | Final Project Documentation

Repository: https://github.com/Meet-Jain-25/StaffSync-HRMS

1. Introduction

Project Title
StaffSync – Modern Human Resource Management Solution

Project Description
StaffSync is a web-based Human Resource Management System (HRMS) developed using the MERN stack. It provides role-based workflows for Human Resources (HR), Managers, and Employees to manage employee records, attendance, leave requests, announcements, and training programs through a centralized application.

Team Members
Team Member: Meet Jain — sole project developer responsible for project planning, implementation, integration, testing, and documentation.

SkillWallet ID- SWUID2026403825

2. Project Overview

Purpose
To provide a centralized platform for employee records, attendance, leave requests, organizational announcements, and training programs, with access controlled by user role.

Implemented features
- Role-based dashboards for HR, Manager, and Employee.
- JWT authentication and role authorization.
- HR employee creation, listing, editing, and deactivation.
- Employee attendance check-in, check-out, and attendance history; HR can view attendance records.
- Employee leave applications and personal leave history; HR/Manager can review requests.
- Training program listing and management.
- Announcements with categories General, HR, Event, Policy, and Urgent.
- Landing, login, and registration interfaces.

Roles
- HR: employee administration, organization-wide attendance view, leave review, announcement management, and training management.
- Manager: leave review, training management, and announcement management.
- Employee: own attendance, leave application/history, and read-only announcements/training access.

3. Architecture

Frontend
React 19 with Vite, React Router, Axios, Bootstrap 5 and Bootstrap Icons, plus custom CSS. API client is in Client/src/services/api.js and uses http://localhost:8000/api. JWT is attached as Authorization: Bearer <token> from localStorage key staffsync_token.

Backend
Node.js with Express 5. The server loads environment variables with dotenv, enables JSON parsing and CORS for http://localhost:5173, and mounts REST route modules under /api.

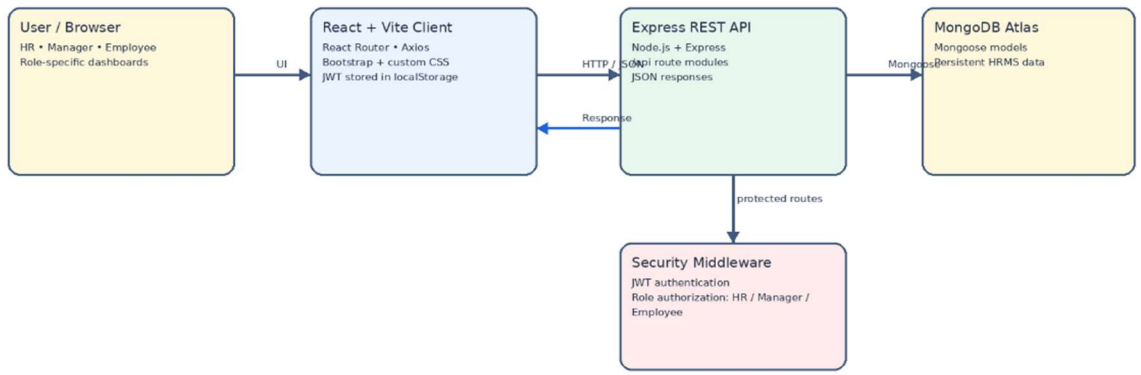
Database

MongoDB Atlas accessed with Mongoose. The connection URI is read from MONGODB_URI. Models: User, Employee, Attendance, Leave, Announcement, and Training.

Request flow

HR/Manager/Employee browser → React UI → Axios → Express route → JWT authentication middleware → role authorization middleware → Mongoose model → MongoDB → JSON response.

StaffSync — System Architecture

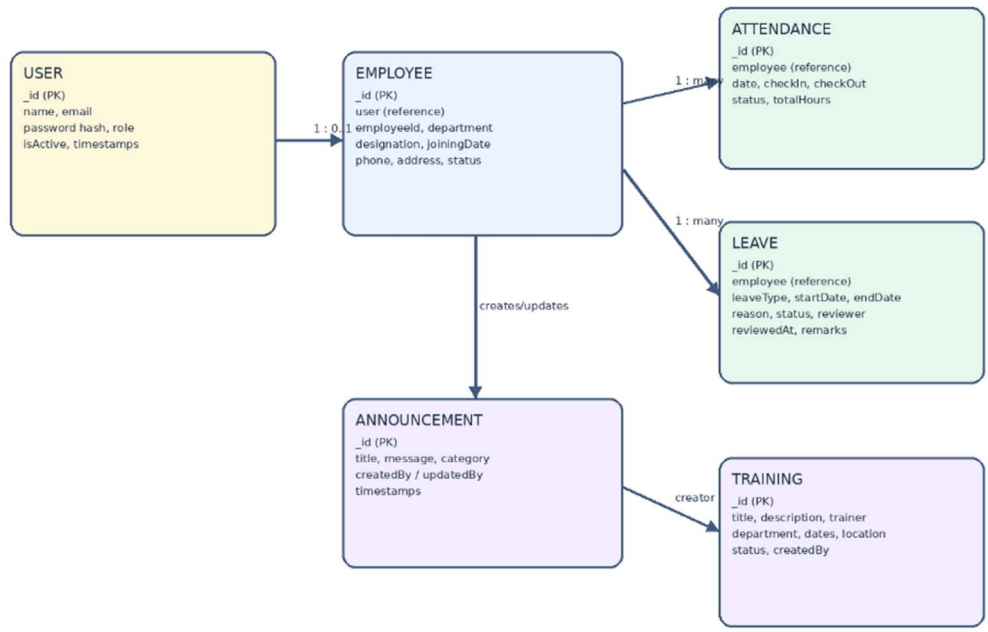


Model overview

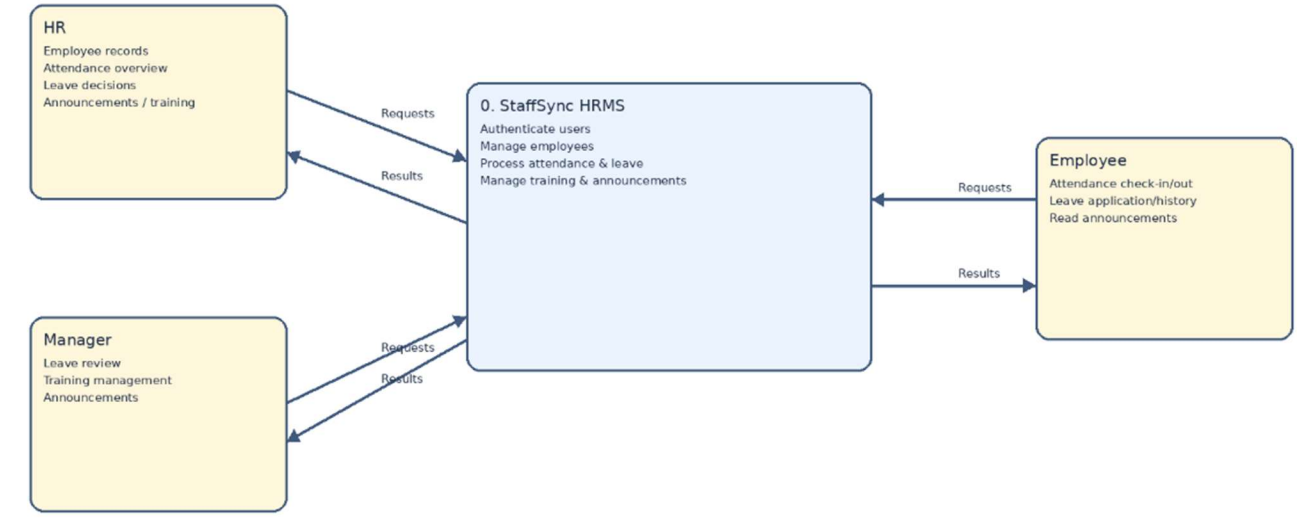
- User: name, email, hashed-password field, role (HR/Manager/Employee), active flag, timestamps.
- Employee: linked User, employeeId, department, designation, joiningDate, phone, address, employmentStatus.
- Attendance: employee, date, checkIn, checkOut, status, totalHours, remarks; unique employee/date index.
- Leave: employee, leaveType, startDate, endDate, reason, status, reviewer, review time, remarks.
- Announcement: title, message, category, creator/updater, timestamps.

Training: title, description, trainer, department, start/end dates, location, status, creator, timestamps.

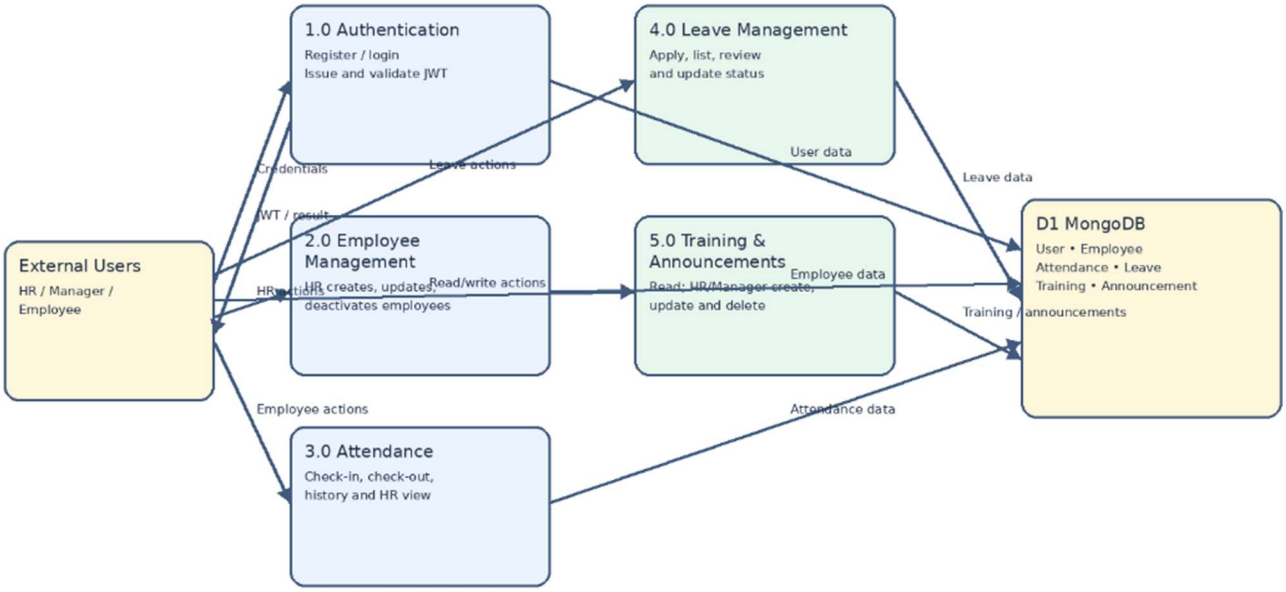
StaffSync — Entity Relationship Diagram

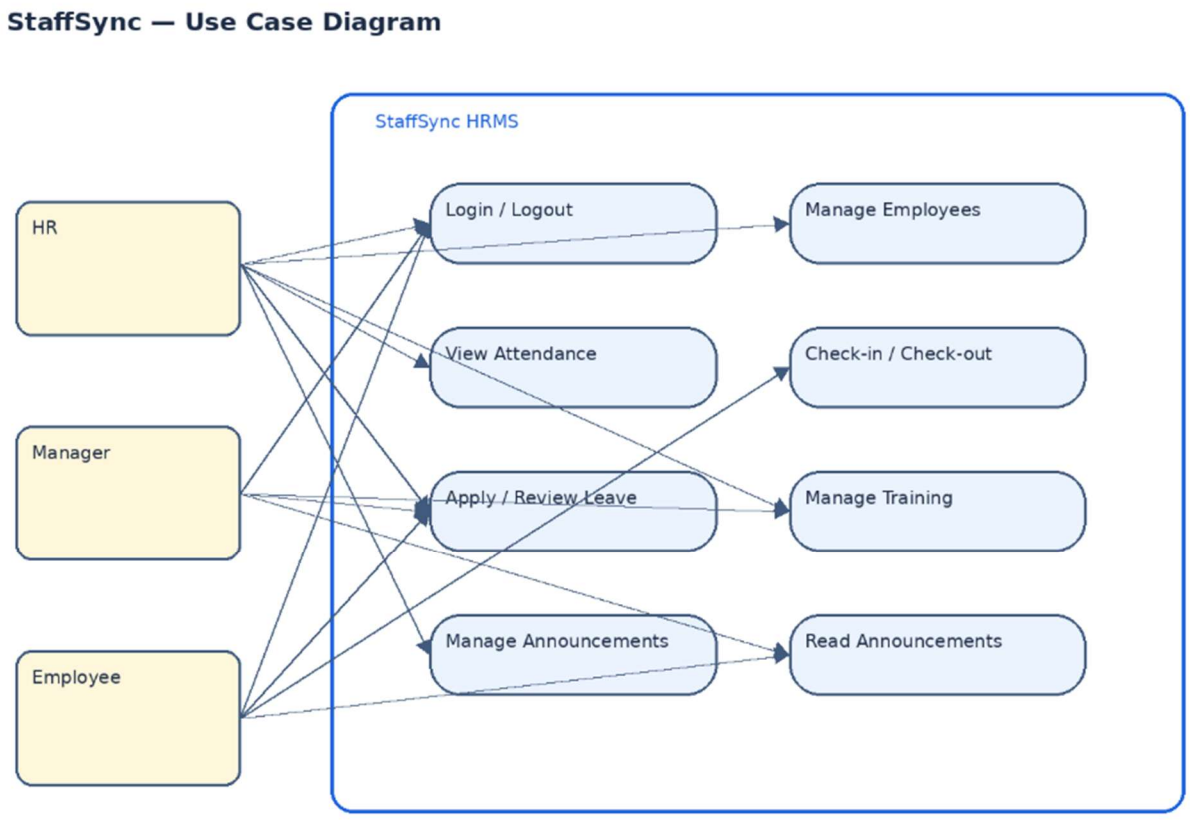


StaffSync — Data Flow Diagram (Level 0)



StaffSync — Data Flow Diagram (Level 1)





4. Setup Instructions

Prerequisites

- Node.js and npm (the development environment previously reported Node v24.21.0 and npm 11.19.0).
- MongoDB Atlas account/database and a valid connection string.
- Git for cloning/version control.
- Code editor such as Visual Studio Code.

Installation

- Clone: git clone https://github.com/Meet-Jain-25/StaffSync-HRMS.git
- Enter repository: cd StaffSync-HRMS
- Enter application: cd StaffSync
- Install backend dependencies: cd Server then npm install
- Create Server/.env based on Server/.env.example. Set MONGODB_URI, JWT_SECRET, and PORT; do not commit real secrets.
- Start backend: npm run dev (or npm start).
- Open a second terminal in StaffSync/Client and run npm install, then npm run dev.

Environment configuration

Server code reads MONGODB_URI, JWT_SECRET, and PORT.

5. Folder Structure

Repository layout

- Phase wise Documentation/ — ideation, requirement analysis, project planning, project design, and testing documents.
- StaffSync/Client/ — React/Vite frontend.
- StaffSync/Server/ — Express API, controllers, middleware, Mongoose models, routes, configuration, and user-creation scripts.

Client structure

- src/components/auth/ — Login.jsx, Register.jsx
- src/components/employee/ — Announcements.jsx, Attendance.jsx, LeaveManagement.jsx
- src/components/hr/ — HRAnnouncements.jsx, HRAttendance.jsx, HRLeaveManagement.jsx
- src/components/manager/ — TrainingManagement.jsx
- src/pages/ — LandingPage.jsx, EmployeeDashboard.jsx, HrDashboard.jsx, ManagerDashboard.jsx
- src/services/api.js — shared Axios client
- App.jsx, App.css, index.css, main.jsx — app shell and styles

Server structure

- config/db.js — MongoDB connection
- controllers/ — announcement, attendance, auth, employee, leave, and training logic
- middleware/authMiddleware.js — JWT protection and role authorization
- models/ — Announcement, Attendance, Employee, Leave, Training, User
- routes/ — corresponding API route modules
- scripts/ — createHRUser.js and createManagerUser.js
- server.js — Express setup, route mounting, and health endpoint

6. Running the Application

Backend

From StaffSync/Server: npm install, then npm run dev. For the normal start script, use npm start.

Frontend

From StaffSync/Client: npm install, then npm run dev. Vite normally serves the app at http://localhost:5173/.

Local endpoints

- Frontend: http://localhost:5173/
- Backend health check: http://localhost:8000/
- API base URL: http://localhost:8000/api

Health response

GET / returns HTTP 200 with { success: true, message: "StaffSync API is running." } when the server is running.

7. API Documentation

Authentication

POST /api/auth/register — register a user; POST /api/auth/login — login; GET /api/auth/me — get authenticated user (JWT required).

Employees (HR only)

GET /api/employees — list employees; POST /api/employees — create employee; PUT /api/employees/:id — update employee; PATCH /api/employees/:id/deactivate — deactivate employee.

Attendance

POST /api/attendance/check-in — Employee check-in; POST /api/attendance/check-out — Employee check-out; GET /api/attendance/my — Employee attendance history; GET /api/attendance — HR attendance view.

Leave

POST /api/leave — Employee applies for leave; GET /api/leave/my — Employee's leave history; GET /api/leave — HR/Manager list leave requests; PATCH /api/leave/:id/review — HR/Manager review decision.

Training

GET /api/training and GET /api/training/:id — authenticated roles can read; POST /api/training, PATCH /api/training/:id, DELETE /api/training/:id — HR/Manager only.

Announcements

GET /api/announcements and GET /api/announcements/:id — authenticated roles can read; POST /api/announcements, PATCH /api/announcements/:id, DELETE /api/announcements/:id — HR/Manager only.

Example request

Employee check-in: POST /api/attendance/check-in with a valid Employee JWT. Example success status previously tested: 201.

8. Authentication

Authentication flow

The server issues/validates JSON Web Tokens (JWT). The React Axios client reads localStorage key staffsync_token and adds it to protected requests as a Bearer token. GET /api/auth/me returns the current authenticated user's id, name, email, and role.

Authorization

The protect middleware verifies the JWT, loads the user, and rejects missing/invalid/expired tokens or inactive accounts. The authorize middleware checks the user's role and returns HTTP 403 for disallowed roles.

Password handling

Passwords are hashed using bcryptjs before being stored in the database. Plain-text passwords are not stored. During authentication, the submitted password is compared with the stored password hash.

Security recommendations

- Use a strong unique JWT_SECRET and keep it in the server environment.
- Do not commit .env files or share live credentials.
- Validate inputs and enforce permissions on the backend.
- Use HTTPS and production-appropriate CORS configuration when deployed.

9. User Interface

Implemented interface areas

- Landing page.
- Login and registration.
- HR dashboard and employee management.
- HR attendance view and leave review.
- Employee dashboard, attendance, and leave management.
- Manager dashboard and training management.
- Announcements management for HR/Manager and read-only announcements for employees.

Screenshot evidence

- Figure 1 — Landing page.
- Figure 2 — Login/registration.
- Figure 3 — HR dashboard and employee list.
- Figure 4 — Employee attendance and leave screen.
- Figure 5 — Leave review screen.
- Figure 6 — Training management.
- Figure 7 — Announcement management and employee view.

Screenshot status

Application and Postman screenshots. The UI screenshots are included in Appendices A and C, and API test evidence is included in Appendix B. The screenshots document the visible state of the application; they are not a substitute for a full regression test.

10. Testing

Tools and approach

Postman is the API testing tool identified by the project. Testing covers successful requests, invalid inputs, and role-based access restrictions.

Previously verified outcomes

- Attendance check-in: 201; duplicate check-in: 409; check-out: 200 with totalHours; own attendance endpoint: 200.
- Announcements: authorized CRUD and category filtering worked; invalid category returned 400; Employee write operations returned 403; deleted announcement lookup returned 404.
- Training: list returned 200; create returned 201; Employee create attempt returned 403.

Final Regression Testing and Results

Regression testing was performed to verify that the implemented features of StaffSync function correctly and that changes to one module do not adversely affect other modules. Testing was carried out using the local web application and Postman API testing tool.

The following test cases were executed successfully:

- **Authentication Testing:** Verified successful login for HR, Manager, and Employee accounts.
- **Role-Based Access Control:** Verified that users can access authorized resources according to their assigned roles and that unauthorized operations are rejected.
- **Employee Management:** Tested employee creation, updating, deactivation, and employee record retrieval.
- **Attendance Management:** Verified employee check-in, check-out, attendance history, and attendance record retrieval.
- **Leave Management:** Tested leave application submission, request retrieval, approval, and rejection workflows.
- **Training Management:** Verified training program creation, retrieval, updating, deletion, and role-based access restrictions.
- **Announcement Management:** Tested announcement creation, retrieval, updating, deletion, category filtering, and access restrictions.
- **Frontend Integration:** Verified that the frontend communicates correctly with the backend APIs and displays the expected results.
- **API Error Handling:** Verified appropriate responses for unauthorized access and invalid requests.
- **Testing Outcome:** All the above test cases passed successfully. The implemented core functionalities of StaffSync are working as expected, and no unresolved issues were identified during the testing performed.

11. Screenshots

Provided Below in the appendices.

Evidence checklist

- Landing page and dashboards.
- Employee management and leave workflow.
- Attendance check-in/check-out.
- Training and announcement screens.
- Postman screenshots showing success and forbidden/error responses.

12. Known Issues

Application issue status

No unresolved runtime bug was established by source inspection alone. The current build should be run and tested before claiming the application has no known issues.

13. Future Enhancements

Suggested future improvements

- Email/in-app notifications for leave decisions and announcements.
- Attendance and leave analytics with downloadable reports.
- Search, filters, pagination, and audit logs.
- Employee document uploads if required by the project scope.

Appendix A — Application Screenshots

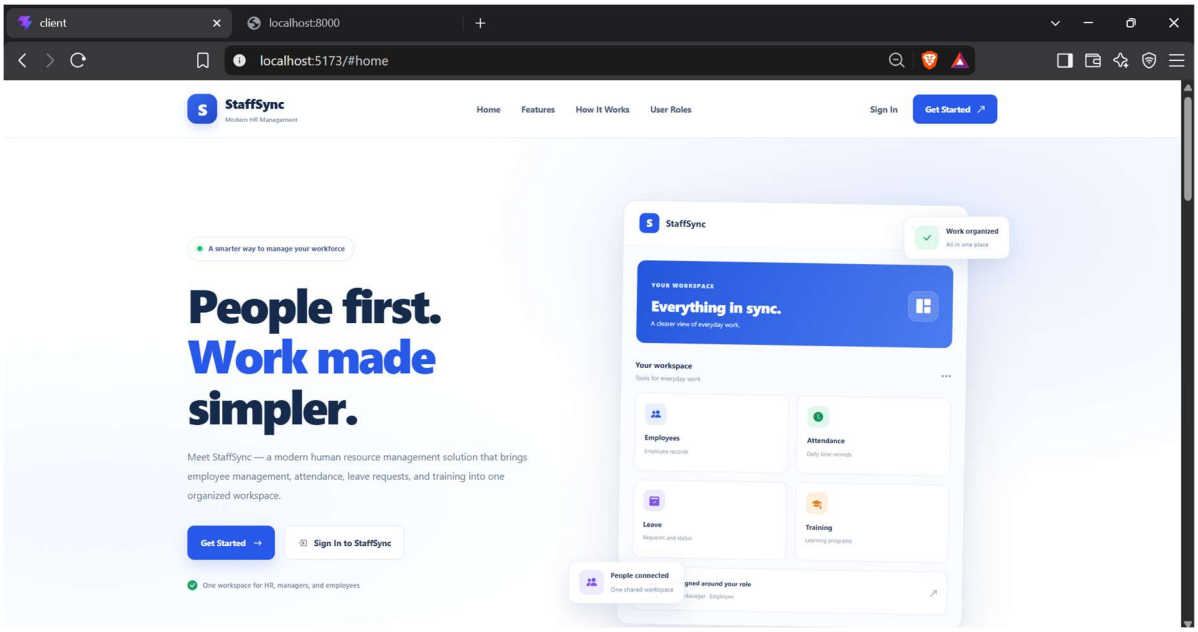


Figure 1. StaffSync landing page

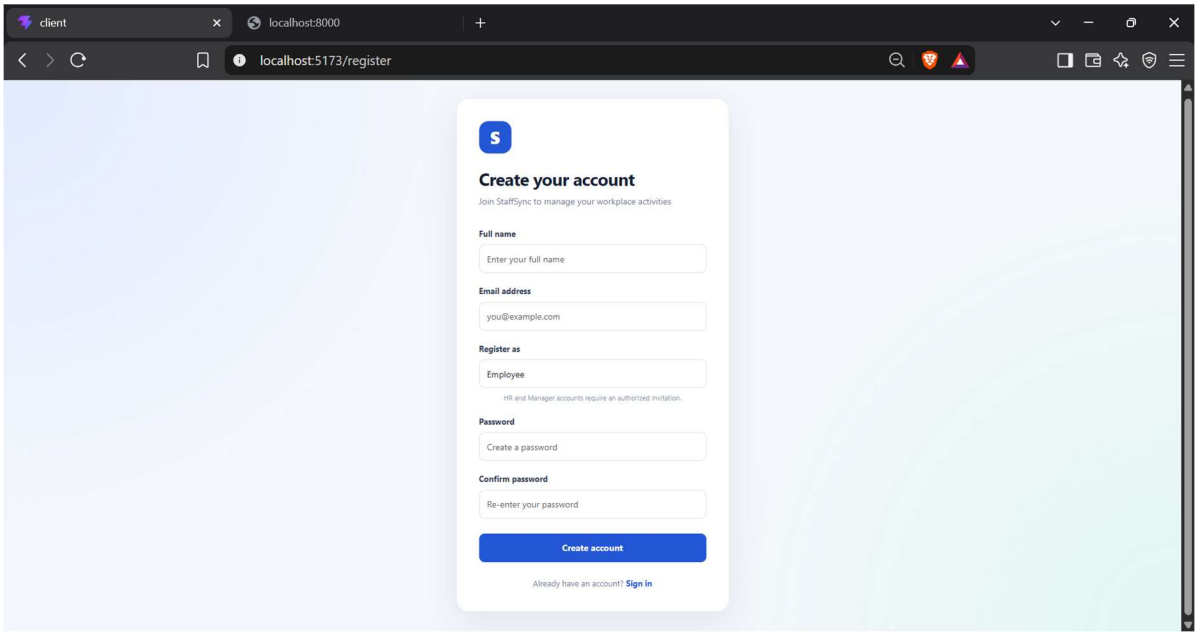


Figure 2. User registration page

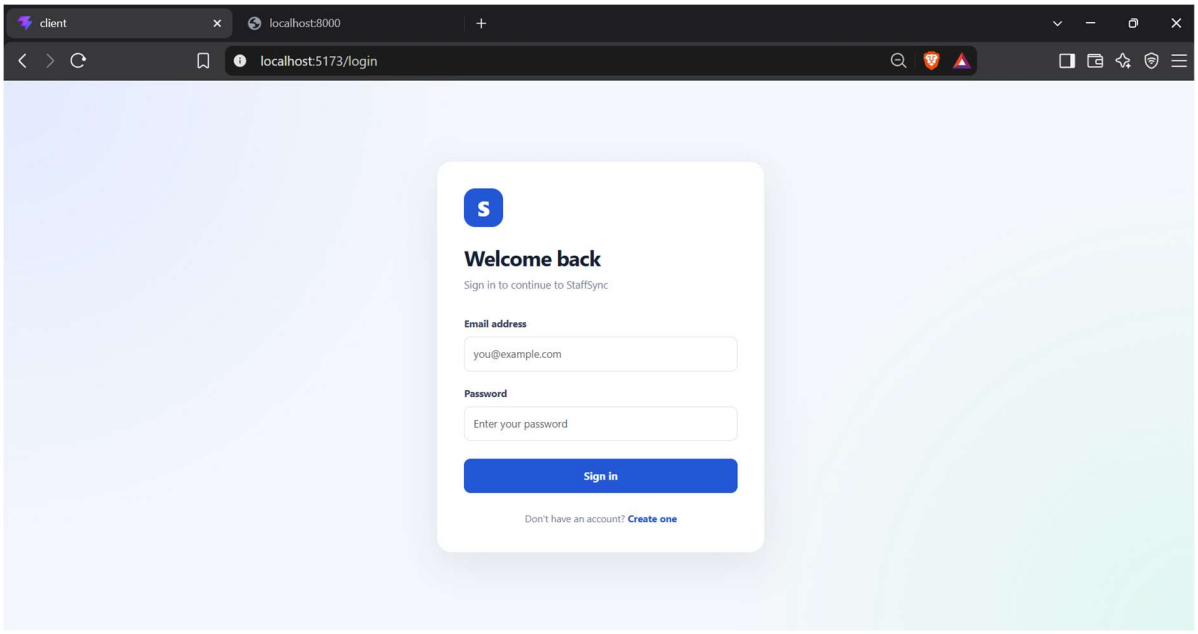


Figure 3. User login page

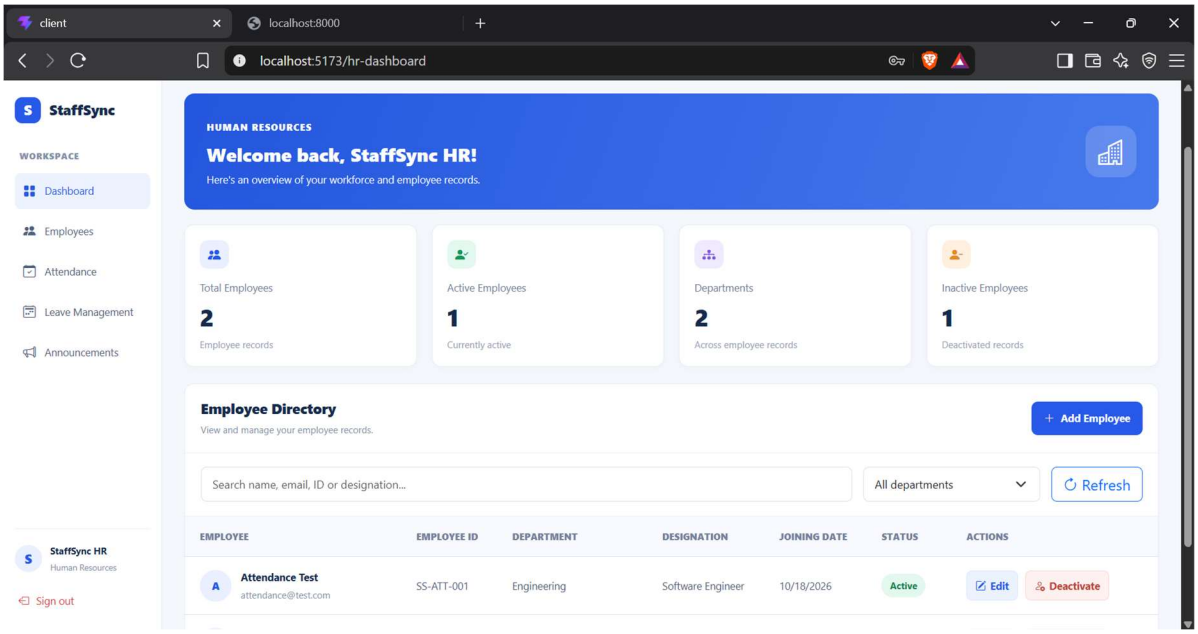


Figure 4. HR dashboard and employee directory

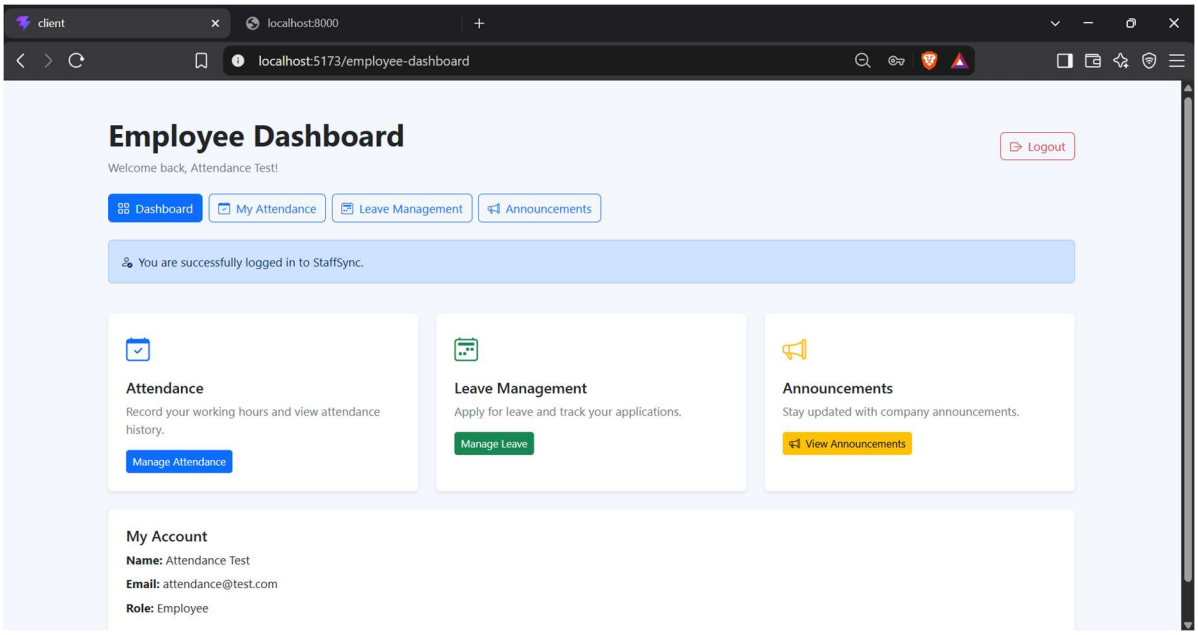


Figure 5. Employee dashboard

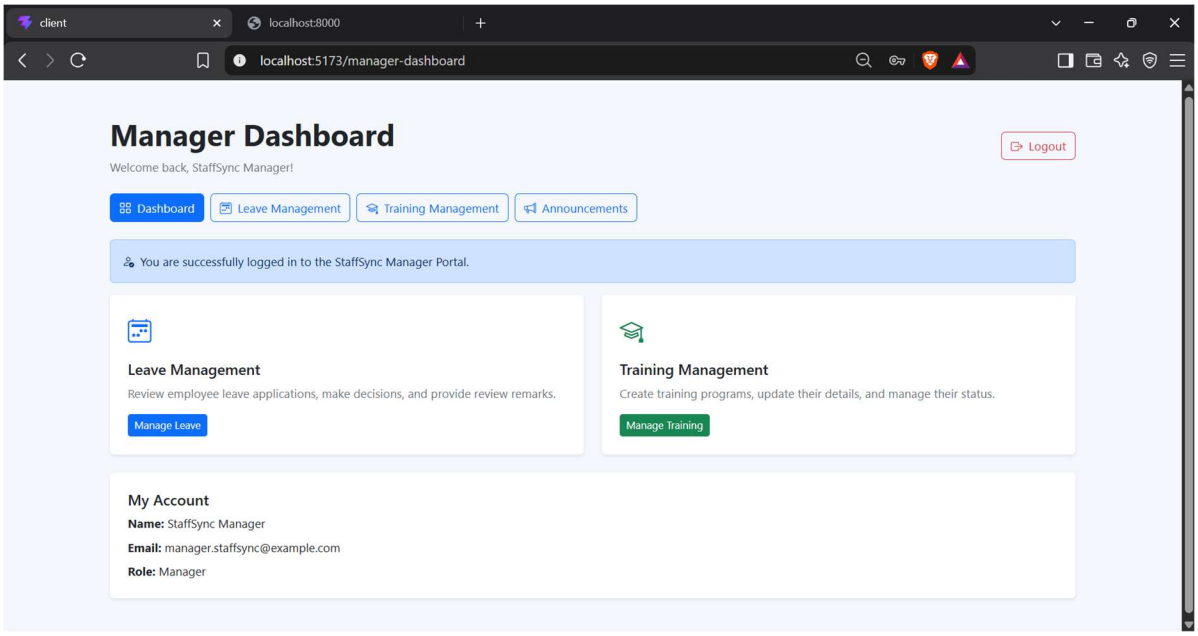


Figure 6. Manager dashboard

Appendix B — Postman API Testing Evidence

The following screenshots show successful employee and HR login, role-based authorization enforcement, and attendance check-in/check-out responses from the local StaffSync API.

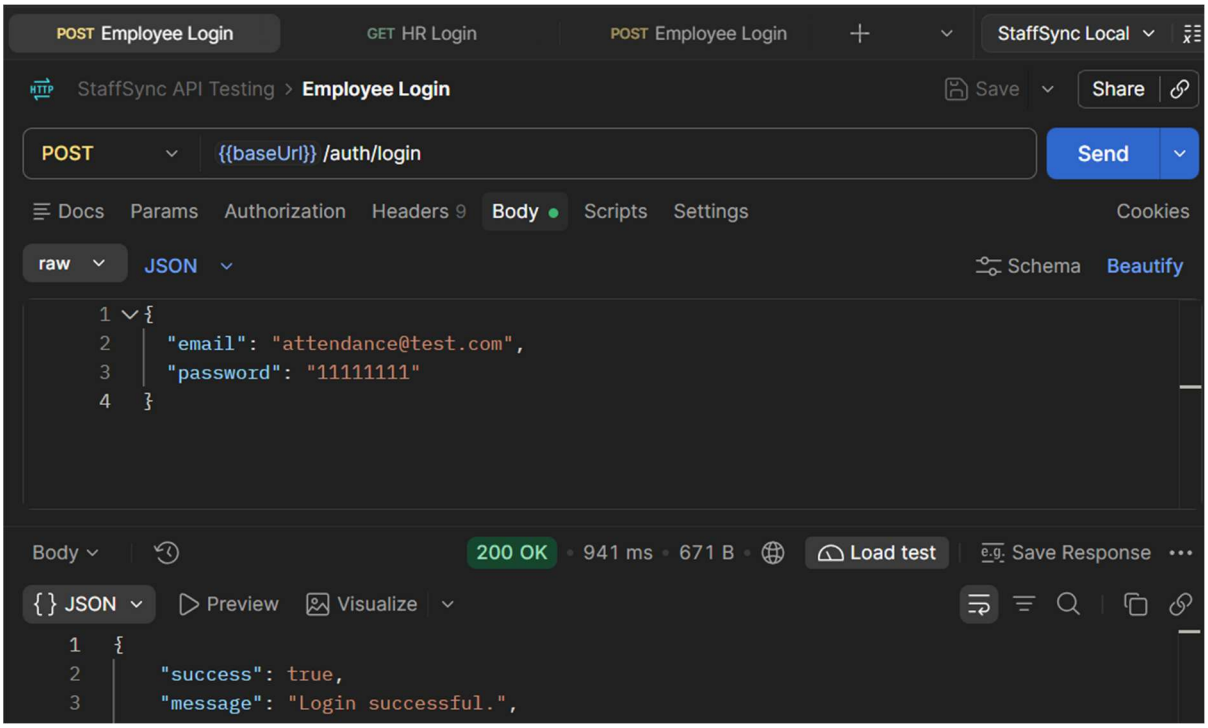


Figure 7. Employee login API — POST /api/auth/login — HTTP 200 OK.

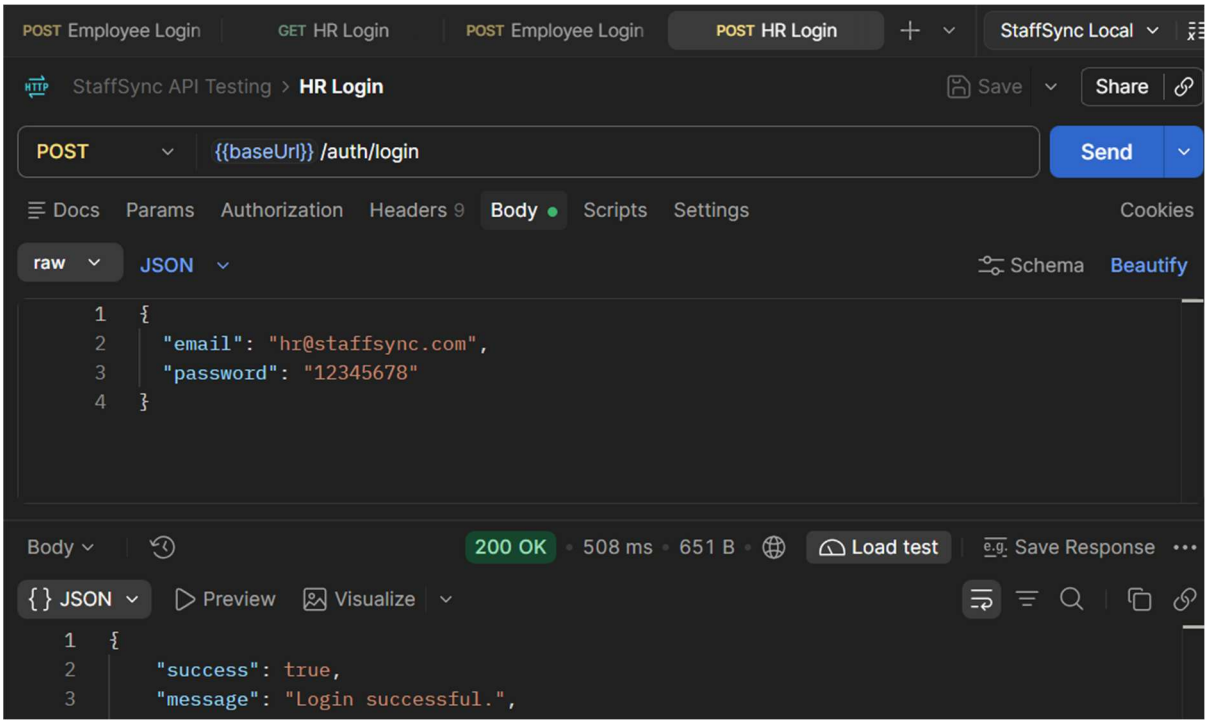


Figure 8. HR login API — POST /api/auth/login — HTTP 200 OK.

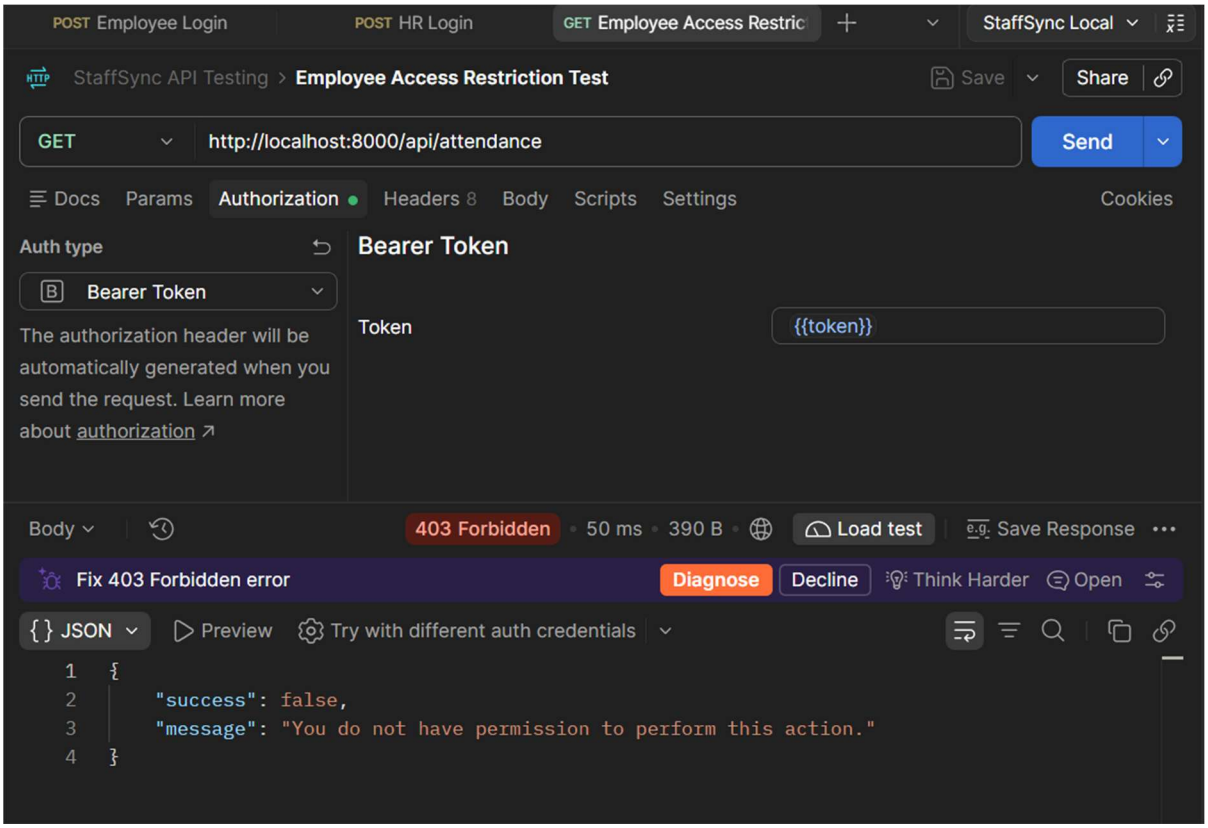


Figure 9. Role authorization test — Employee token denied access to GET /api/attendance — HTTP 403 Forbidden.

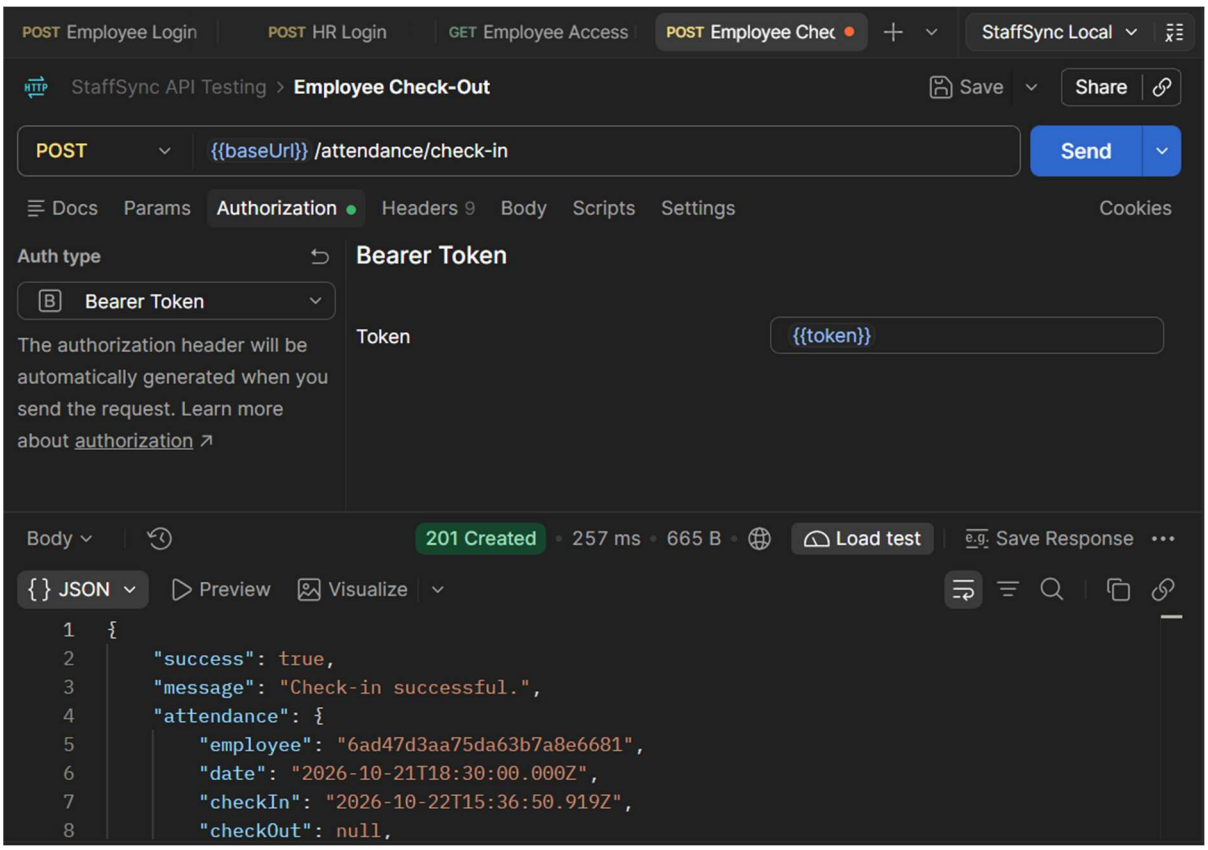


Figure 10. Employee attendance check-in — HTTP 201 Created.

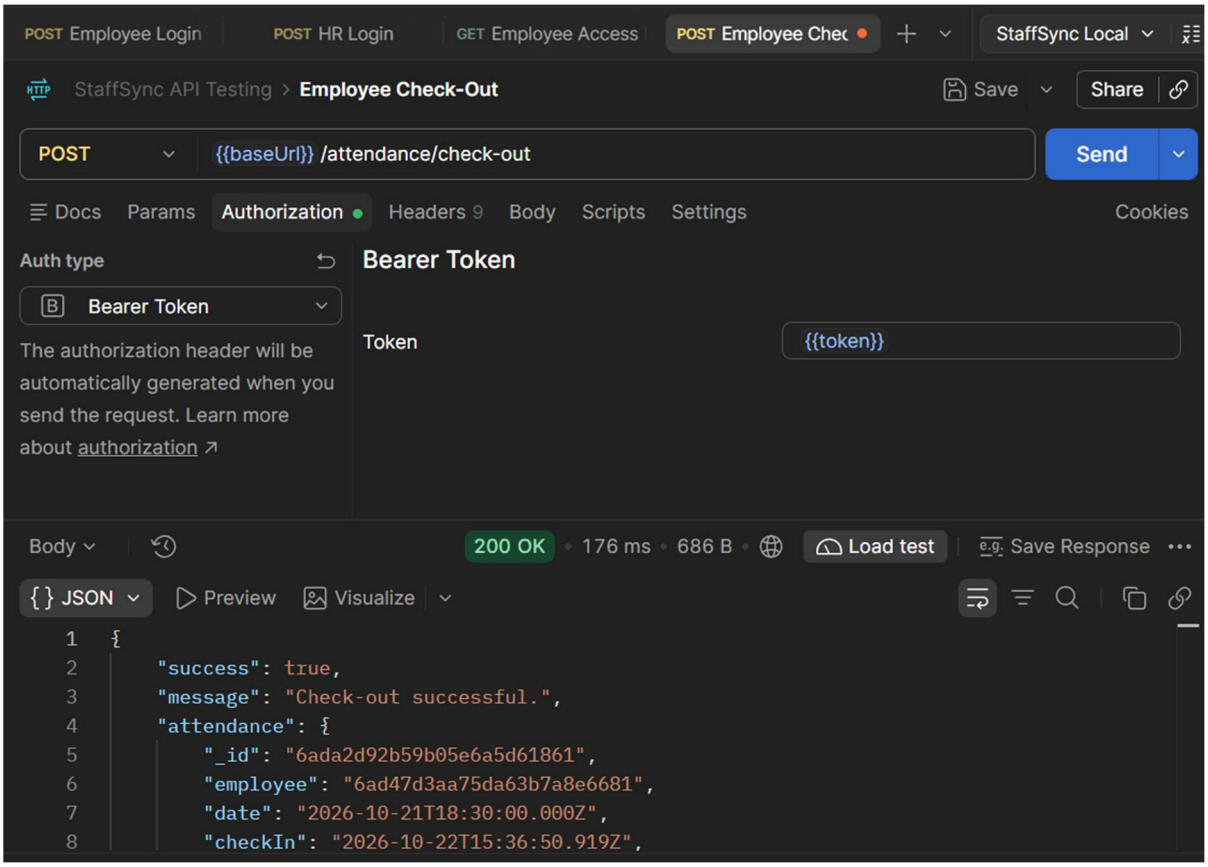


Figure 11. Employee attendance check-out — HTTP 200 OK.

Appendix C — Feature Screenshots

This appendix contains additional screenshots captured from the running local StaffSync application. They are grouped by feature and role to support the User Interface, project functionality, and testing sections.

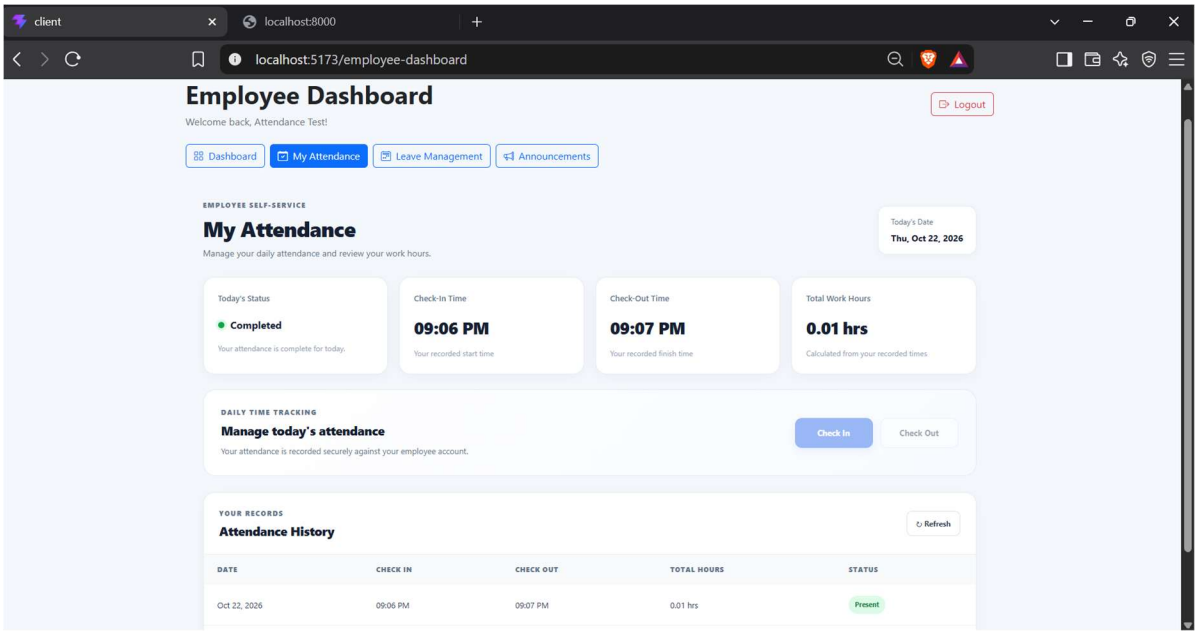


Figure 12. Employee self-service attendance page showing daily status, check-in/check-out times, total hours, and attendance history.

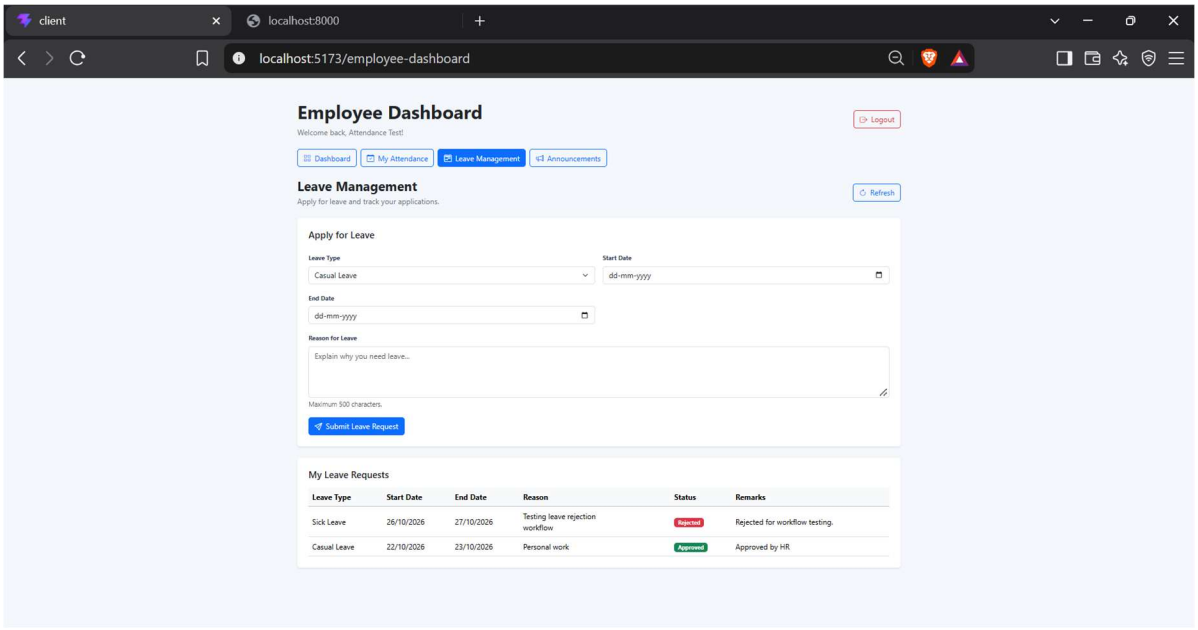


Figure 13. Employee leave management page showing the leave application form and the employee's leave request history.

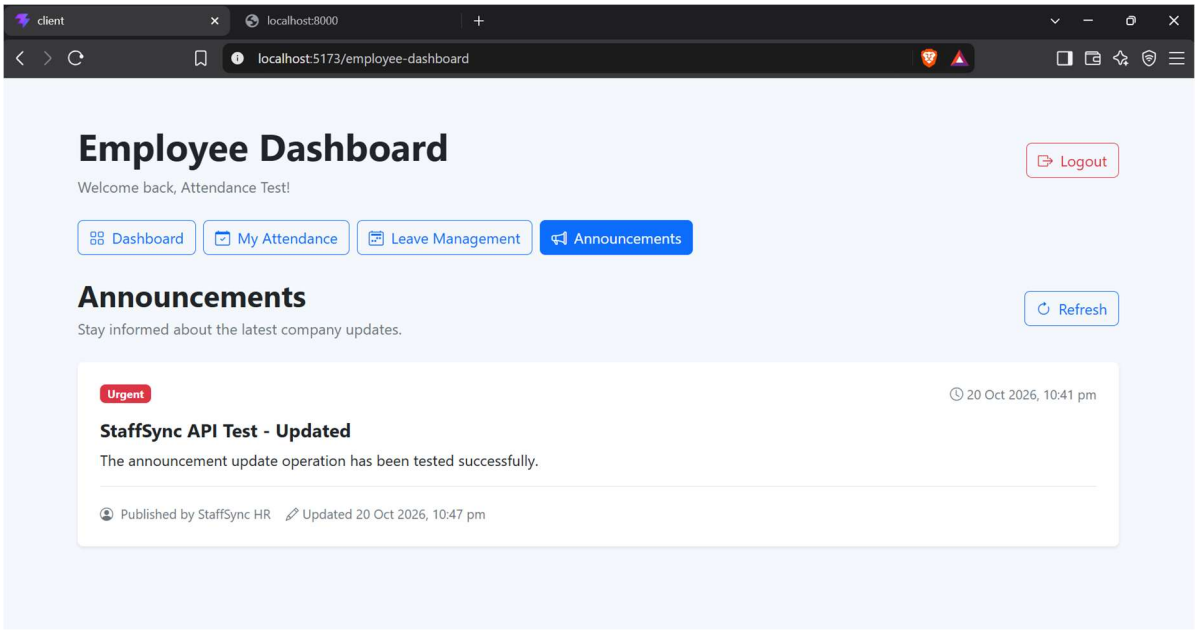


Figure 14. Employee announcements page showing a published announcement in the read-only employee view.

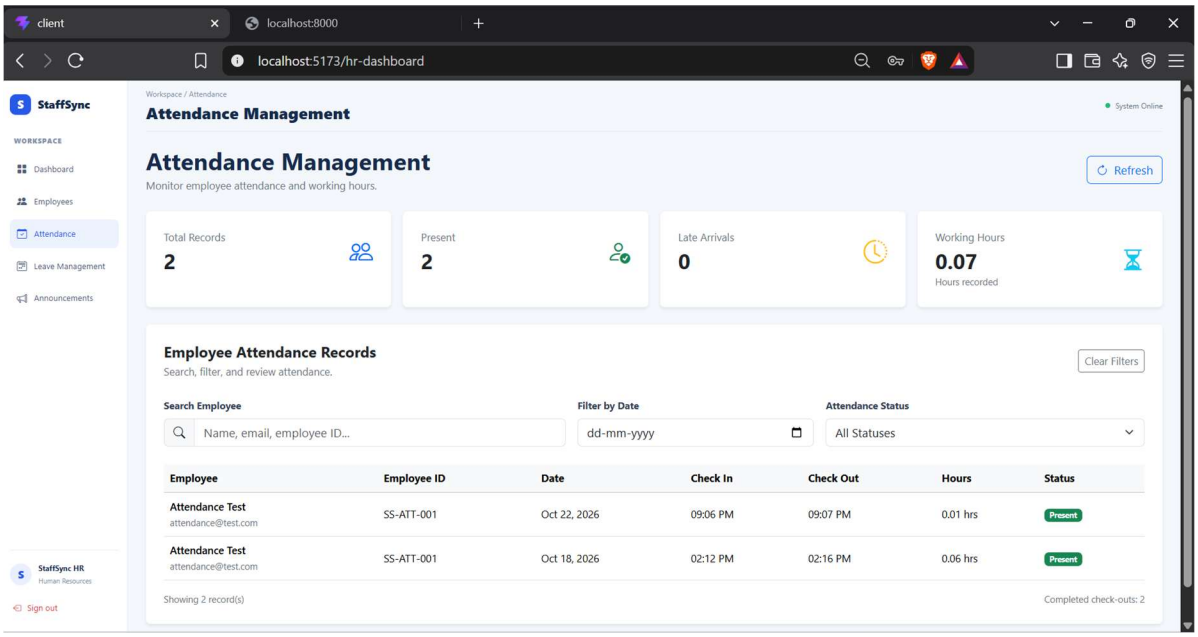


Figure 15. HR attendance management page showing attendance summaries, filters, and employee attendance records.

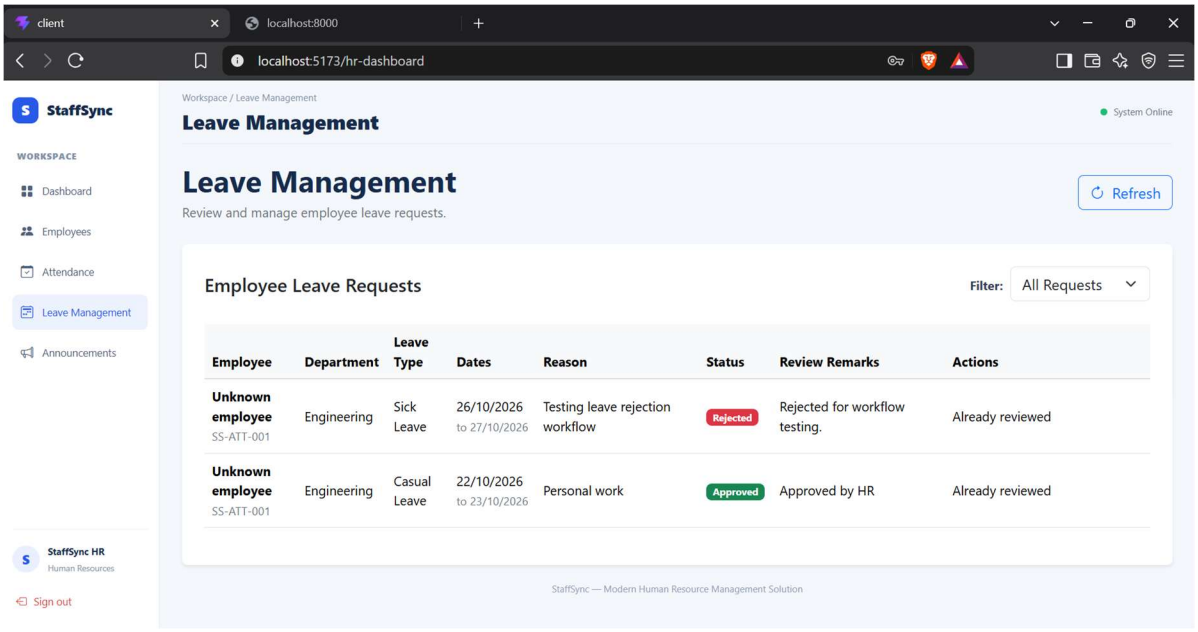


Figure 16. HR leave management page showing employee leave requests, status, review remarks, and actions.

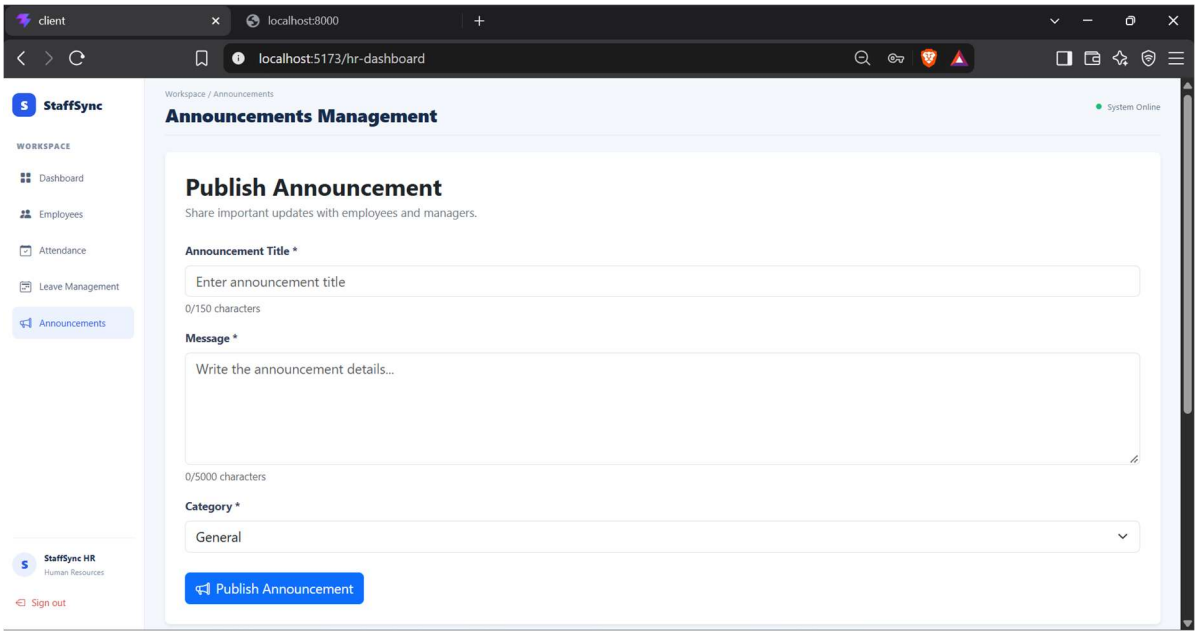


Figure 17. HR announcements management page showing the announcement publishing form and category selection.

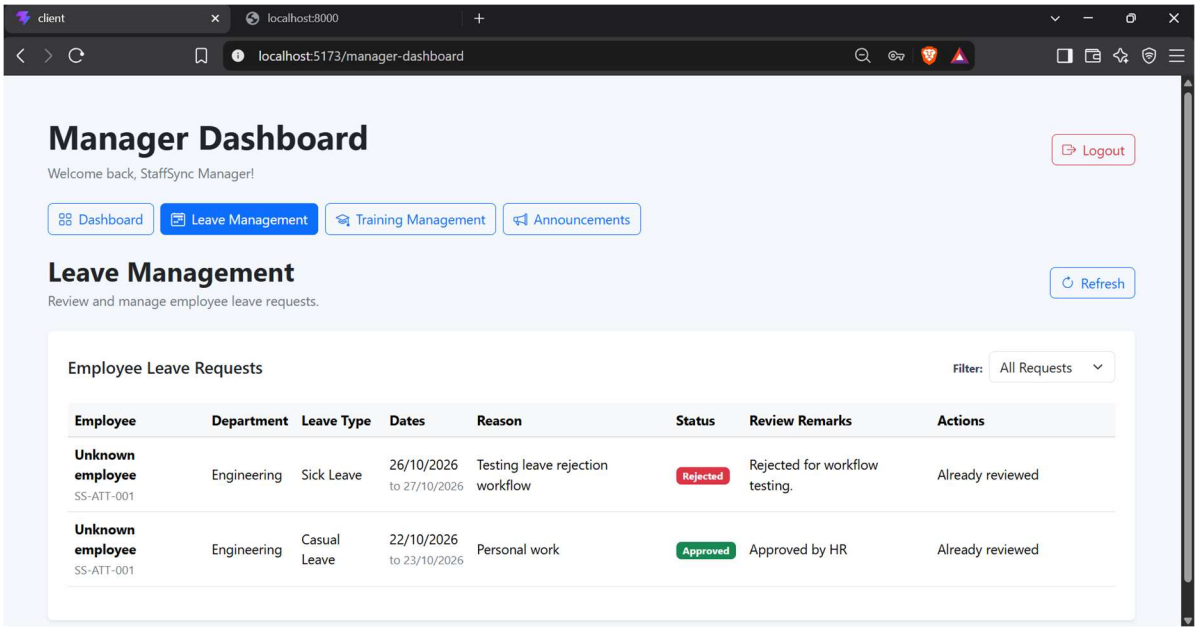


Figure 18. Manager leave management page showing reviewed employee leave requests.

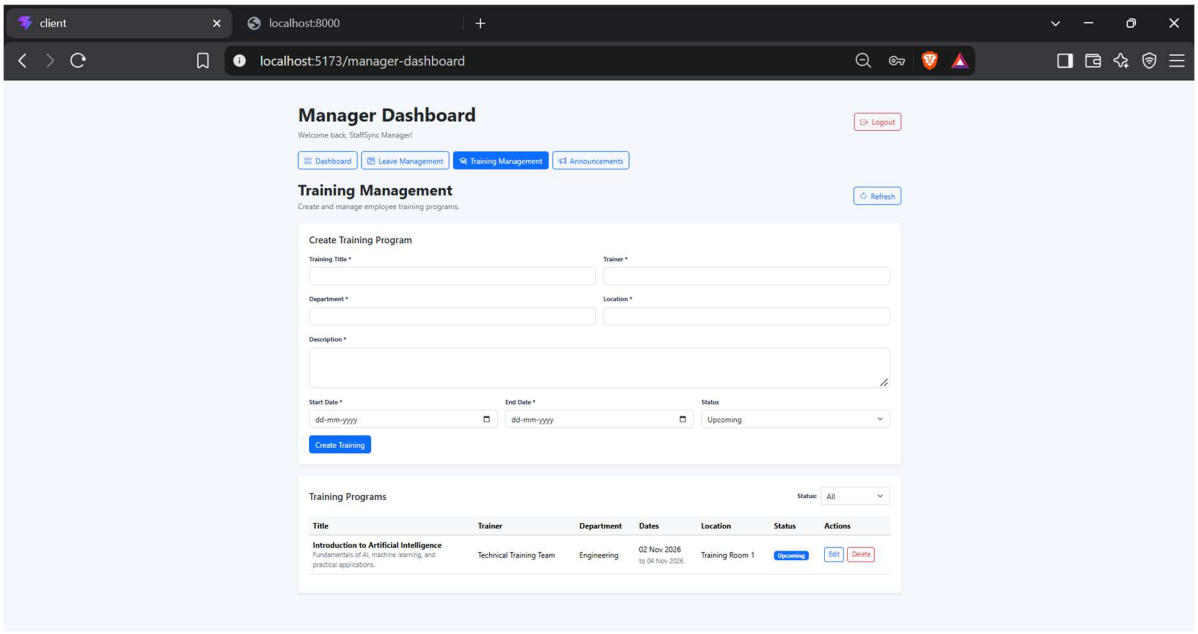


Figure 19. Manager training management page showing the training program form and existing training program list.

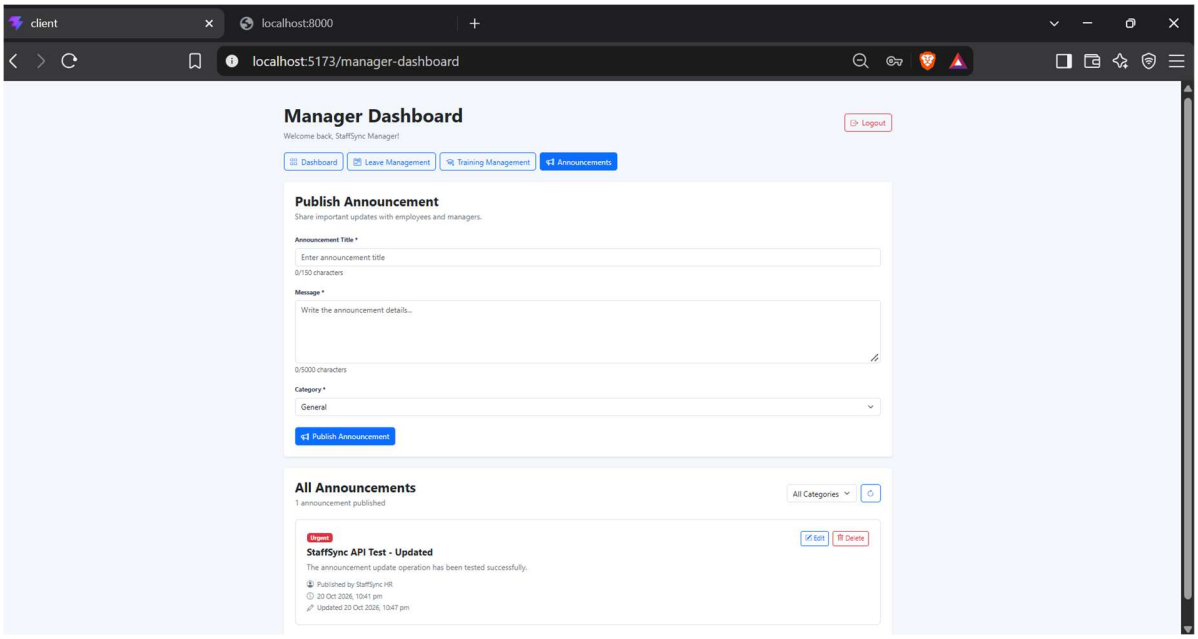


Figure 20. Manager announcements management page showing publishing controls and the announcements list.